

TASK1

Shortlisting Task

Task A — Mini Hospital Management System (HMS)

Python Serverless Backend Track

Department	Engineering — Backend
Work window	~ 2 days from the date this task is released to you
Deadline	Hard cutoff — communicated at task release with exact date and time
Submission	GitHub repo link + video link — both submitted via Google Form (link sent with task)

This task is designed to be unfamiliar. There is no template. We are not looking for the right answer — we are reading how you think and communicate when there is no instruction manual. AI tools are allowed and encouraged, on the condition that you can defend every line of what you submit. The deadline is hard. Submit on time.

What You Are Building

Build a small hospital management web application focused on doctor availability and patient appointment booking, with a separate serverless email notification service.

The system runs fully locally. No cloud deployment is required. The focus is on code structure, working integrations, and clean implementation — not production deployment.

Default Tech Stack

You may use whatever fits your setup. The default stack:

- Backend Framework: Django
- Database: PostgreSQL — installed locally
- ORM: Django ORM
- Auth: Username/email + password, session-based
- Serverless Email Service: Serverless Framework (serverless.yml) with serverless-offline for local testing

User Roles

Doctor

- Can sign up and log in
- Has a doctor dashboard
- Can set and update their own availability time slots (e.g. 10:00-11:00, 11:00-11:30)
- Can only see and manage their own availability and bookings — not other doctors'

Patient

- Can sign up and log in
- Has a patient dashboard
- Can view doctors and their available time slots
- Can book one available time slot with a doctor
- Once a slot is booked, it is blocked — no other patient can book the same slot

Core Functional Requirements

Authentication

- Sign up and login for both doctors and patients
- Passwords stored securely — hashed
- Role-based access: patients cannot access doctor-only actions, and vice versa

Doctor Availability

- Doctor can create availability slots by date and time
- Slots are visible to patients only if they are in the future and not already booked

Booking Flow

Patient selects a doctor, a date, and a time slot. The system must:

- Verify the slot is still free — handle race conditions (two patients attempting to book the same slot simultaneously)
- Create the booking
- Mark the slot as booked so no other patient can take it

Google Calendar Integration

When a booking is confirmed, create a calendar event in both the doctor's Google Calendar and the patient's Google Calendar.

You can assume one Google account per user. Use Google Calendar API with OAuth2.

Event details:

- Title: 'Appointment with Dr. <DoctorName>' on the patient's calendar / 'Appointment with <PatientName>' on the doctor's calendar
- Start and end time = the selected slot
- Optional: short description

Email Notification via Serverless Function

Build a separate Python serverless function using the Serverless Framework (serverless.yml). This function's only job is to send emails.

It must support at least two triggers:

- SIGNUP_WELCOME — send a welcome email on sign up
- BOOKING_CONFIRMATION — send a confirmation email when a booking is made

Your HMS Django backend calls this function via its HTTP endpoint. Use serverless-offline to run and test the function locally. Actual AWS deployment is optional — the focus is on structure and working local integration.

Emails can be sent via Gmail SMTP or any other SMTP provider.

Offline and Local Demo

Your system must be fully runnable locally:

- Main Django app running locally
- Serverless email service running via serverless-offline
- All features demonstrable without any cloud deployment

The Design Decision Requirement

Your README must include a section where you name one hard design decision you made — something where two reasonable approaches existed and you had to choose one.

It can be anything real from the build: how you handled the race condition in slot booking, how you enforced role-based access, how the serverless function connects to Django, how OAuth2 tokens are stored and refreshed for calendar access.

That section must do three things:

- Name the problem clearly
- Explain both approaches you considered
- Defend the one you chose — not just describe it, defend it

A submission without a named and defended design decision will be treated as incomplete regardless of how well the rest of the system works.

Submission Format

Two things to submit. Both go through the Google Form link sent alongside this task.

1. Public GitHub Repository

One public repository with the full codebase. Separate the Django app from the serverless function clearly. Structure it like this:

```
your-repo/ |— README.md                                <- written report (see below) |—
ai-tool-usage-log/                                <- all AI chat threads go here |—
chatgpt-thread-1.md |— claude-session.txt |— (any format, any number of
files) |— hms/                                <- Django application |— ... |—
email-service/                                <- serverless email function |— serverless.yml |
|— handler.py |— ... |— requirements.txt
```

Submit the public repository link via the Google Form.

2. Video Demo

A 10-minute screen recording that demonstrates all features and walks through the code.

The video must show:

- Doctor sign up and login — setting availability slots
- Patient sign up and login — viewing doctors, selecting a slot, booking it
- Slot blocking — another patient cannot book the same slot after it is taken
- Google Calendar events created for both doctor and patient after a confirmed booking
- Serverless email function triggered — SIGNUP_WELCOME and BOOKING_CONFIRMATION emails sent via serverless-offline
- Code walkthrough — not line by line, but enough to show you understand what you built and why
- One design decision explained in your own words

Upload to Google Drive. Set sharing to 'Anyone with the link can view'. Submit the link via the Google Form.

A broken or private Google Drive link at the time of evaluation is treated the same as a missing video.

README.md — The Written Report

Your README is your written report. Use these exact section headings, in this order:

Setup and Run

Step-by-step instructions to run the full system locally — Django app and serverless-offline together. Must work on a fresh machine. We will run it.

System Architecture

How the Django app and the serverless email service connect. Your data model decisions. How role-based access is enforced. How the Google Calendar integration is structured.

The Design Decision

One hard call you made. Both options. Your choice. Your reasoning. A position, not 'it depends'.

Limitations

What would break in production. What you would fix first and why.

ai-tool-usage-log/ Folder

Every AI tool interaction used during this task goes in this folder. ChatGPT, Claude, Copilot, Codex — any tool. Export or copy-paste the threads as .txt or .md files. Name files clearly. Do not clean them up.

The ai-tool-usage-log/ folder is required. If you used no AI tools at all, add one file in the folder stating that. A missing folder is treated as an incomplete submission.

Submission Checklist

☐	Public GitHub repository — repo is set to public
☐	GitHub repo link submitted via Google Form
☐	README.md with all four sections: Setup and Run, System Architecture, The Design Decision, Limitations
☐	ai-tool-usage-log/ folder present with thread files (or a note if no AI used)
☐	Django app runs locally from README instructions
☐	Serverless email service runs locally via serverless-offline
☐	Doctor and patient roles both functional with correct access restrictions
☐	Doctor can create and manage their own availability slots
☐	Patient can view available slots and book one
☐	Slot blocking works — booked slot is unavailable to other patients
☐	Race condition in booking is handled
☐	Google Calendar events created for both doctor and patient on booking confirmation
☐	SIGNUP_WELCOME email trigger works via serverless-offline
☐	BOOKING_CONFIRMATION email trigger works via serverless-offline
☐	Video uploaded to Google Drive — sharing set to 'Anyone with the link can view'
☐	Google Drive video link submitted via Google Form

The deadline is hard. A late submission without a message sent before the deadline is not reviewed. If something comes up, send a message before the cutoff — not after.